Plan d’implémentation — M2 Comptes utilisateurs

> Version 1.3 — 24 octobre 2026
> Statut : M2 implémenté, vérifié techniquement et validé par le demandeur le 24 octobre 2026 ; e-mail de test reçu.
> Projet : Academia — Bibliothèque des Épreuves ESGC VAK.

1. Identification

Élément	Valeur
Module	M2 — Comptes utilisateurs
Auteur	Équipe Academia, à confirmer
Validateur métier	Demandeur / responsable désigné, à confirmer
Date	24 octobre 2026
Version	1.3
Statut	Module validé par le demandeur le 24 octobre 2026

Ce plan applique la trame du guide « Avant de coder un module » au rôle attribué à M2 dans le document d’architecture : gérer le cycle de vie des comptes utilisateurs et l’affectation des rôles. Les décisions métier prises pendant le cadrage sont consignées ici et dans le plan M1 ; les points restant à confirmer sont listés en section 8.

2. Contexte, problème et objectifs

La plateforme doit permettre à la personne habilitée de tenir à jour les comptes nécessaires à l’accès à la Bibliothèque des Épreuves. Un compte doit être rattaché au bon rôle et ses accès doivent refléter les changements apportés à son état ou à son rôle.

Parcours visé : un administrateur authentifié ouvre la gestion des comptes, recherche ou sélectionne un utilisateur, crée ou met à jour son compte et lui affecte un rôle autorisé. Le serveur contrôle chaque action et conserve l’intégrité des données. Les utilisateurs concernés peuvent ensuite s’authentifier par M1.

Objectifs approuvés et implémentés :

- Gérer les comptes : consultation, création, modification, activation/désactivation, suppression logique et restauration.
- Affecter à un utilisateur uniquement un rôle prévu par la matrice d’accès approuvée.
- Réserver les opérations de gestion aux utilisateurs possédant la permission serveur `users.manage`.
- Ne pas recréer l’authentification, les sessions, les rôles ou les permissions déjà pris en charge par M1 et les mécanismes existants.
- Fournir des retours compréhensibles et ne jamais exposer de mot de passe, jeton ou donnée sensible.

Histoires utilisateur

- Comme administrateur autorisé, je veux créer et mettre à jour les comptes afin de donner aux utilisateurs l’accès correspondant à leur fonction.

- Comme administrateur autorisé, je veux affecter un rôle approuvé afin que les permissions de l'utilisateur soient cohérentes avec la matrice d'accès.
- Comme administrateur autorisé, je veux retrouver un compte afin de le modifier sans devoir parcourir une liste non filtrée.
- Comme utilisateur, je veux que les changements d'état ou de rôle de mon compte soient appliqués de manière prévisible à mes accès.

3. Périmètre

Inclus dans M2

- Gestion administrative des comptes utilisateurs.
- Consultation, recherche et pagination des comptes.
- Création et modification des informations de compte (nom et adresse e-mail).
- Affectation d'un rôle existant parmi les rôles approuvés.
- Activation/désactivation et suppression logique restaurable d'un compte.
- Autorisation serveur de chaque écran et opération de gestion.
- Tests des validations, des accès et des effets des changements de rôle ou d'état.
- Coordination avec M1 pour la création du compte, l'authentification, la réinitialisation du mot de passe et l'effet d'une désactivation.

Hors périmètre de M2

- Connexion, déconnexion, sessions, limitation des tentatives et réinitialisation de mot de passe : M1.
- Définition de nouveaux rôles, création ou édition des permissions et modification de la matrice d'accès : hors de la gestion courante des comptes et à livrer comme évolution distincte.
- Gestion des filières, niveaux, matières ou autres référentiels : M3.
- Inscription publique : le comportement reste dans le parcours existant de M1 ; tout nouveau compte reçoit le rôle `etudiant`. Cette décision du 24 octobre révise l'ancienne décision M1 D02.
- Gestion des profils pédagogiques (classe, filière, matières, rattachement d'un responsable) : à coordonner avec M3 et les besoins métier ; aucun champ de ce type n'est établi ici.
- Suppression définitive, import massif, gestion des rôles/permissions, gestion multi-établissement : hors périmètre retenu. La suppression logique et l'audit des changements sensibles sont inclus.

4. Existant, stack et contraintes

Les informations ci-dessous décrivent l'état actuel après l'implémentation M2 et distinguent les éléments préexistants de ceux ajoutés par le module.

Élément	État observé	Conséquence pour le plan
Backend	Laravel `^13.17` déclaré ; PHP `^8.3` déclaré. Le plan M1 consigne Laravel 13.36.0 et PHP CLI 8.4.22.	Reprendre les versions réellement installées et configurées au moment de l'implémentation.
Interface	Inertia, Vue 3 et TypeScript déclarés ; M1 décrit une interface Inertia/Vue.	Suivre les composants, conventions et client de requêtes existants ; ne pas basculer vers Blade/Bootstrap proposés dans le document d'architecture initial.
Base de données	Application et tests configurés sur MySQL ; PHPUnit utilise la base dédiée `academia_test`.	Garder cette base isolée des données de développement et de production.

Élément	État observé	Conséquence pour le plan
Utilisateur	`users` existait avec `name`, `email` unique, `email_verified_at` nullable, `password`, `remember_token` et horodatages ; la migration M2 ajoute `is_active` et `deleted_at`.	Ne pas recréer `users`. Les champs de profil gérés par M2 sont limités à `name` et `email`.
Modèle `User`	Utilise `HasRoles` de Spatie Permission et `SoftDeletes`. `password` et `remember_token` sont masqués ; `is_active` est casté en booléen et vaut vrai par défaut.	Réutiliser les protections et les relations du modèle ; vérifier la sérialisation avant d'exposer un utilisateur à Inertia.
Rôles et permissions	Tables Spatie créées par migration ; `AccessControlSeeder` définit `etudiant`, `responsable_classe`, `administration`, `administrateur` et leurs permissions.	M2 attribue les rôles existants ; ne pas dupliquer les tables, les rôles ou la matrice.
Permission de gestion	`users.manage` est accordée au rôle `administrateur` par le seeder.	Exigée côté serveur sur chaque action M2 ; le masquage d'un bouton ne constitue pas un contrôle d'accès.
Authentification et inscription	Les routes de connexion et d'inscription sont dans `routes/web.php`. `RegisterStudent` crée un utilisateur et lui attribue `etudiant`.	Inscription publique conservée conformément à D11 ; M2 permet à l'administrateur de remplacer le rôle unique.
Interfaces de gestion	Liste Inertia/Vue et routes M2 ajoutées sous `/admin/utilisateurs`.	Garder les mutations sous `auth` et `can:users.manage` ; aucune seconde API ou authentification n'est créée.
Tests	`AuthorizationTest` et `AuthenticationTest` couvrent M1 ; `UserManagementTest` couvre les opérations M2 sur MySQL.	La configuration PHPUnit utilise le schéma de test MySQL dédié `academia_test`, jamais le schéma de développement.

Principaux fichiers concernés par M2

- `routes/web.php`
- `app/Models/User.php`
- `app/Actions/Auth/` et `app/Http/Controllers/Auth/` pour ne pas dupliquer le flux existant
- `app/Http/Controllers/`, `app/Http/Requests/` et éventuellement `app/Policies/`
- `database/migrations/`, `database/seeders/AccessControlSeeder.php`
- `resources/js/pages/` et composants Inertia/Vue existants
- `tests/Feature/AuthorizationTest.php` et nouveaux tests de gestion des comptes

5. Règles métier et autorisations

R01 à R12 appliquent les décisions approuvées le 24 octobre 2026 et le cadre d'accès de M1. R10 précise les contrôles de sécurité nécessaires. La durée de conservation des audits est fixée à 12 mois.

ID	Règle vérifiable	État
R01	Toute route et toute action de gestion de compte exigent une authentification M1 et la permission `users.manage`. Un utilisateur authentifié sans cette permission reçoit un refus serveur.	Appliqué aux routes M2 par `auth` et `can:users.manage`.
R02	Une affectation ne peut utiliser qu'un rôle existant et autorisé dans la matrice approuvée : `etudiant`, `responsable_classe`, `administration` ou `administrateur`. Une valeur envoyée par le client ne crée jamais de rôle ni de permission.	Liste fermée validée côté serveur ; l'affectation remplace tous les rôles précédents.
R03	L'identité de connexion est l'adresse e-mail, qui reste unique. Une modification d'e-mail est refusée si elle est déjà utilisée par un autre compte.	Contrainte unique et validation serveur ; comportement testé.

ID	Règle vérifiable	État
R04	Les mots de passe ne sont ni affichés ni retournés à l'interface. La création administrative utilise un mot de passe aléatoire non communiqué, puis le lien M1 de définition/réinitialisation ; le premier administrateur choisit son secret dans la commande de configuration.	Hachage Laravel ; aucune valeur de mot de passe par défaut n'est codée.
R05	Un compte ne peut pas être supprimé ni désactivé par une opération non autorisée ou par une simple modification de l'interface.	Toutes les mutations sont contrôlées côté serveur ; les suppressions sont logiques et réversibles.
R06	La gestion des comptes ne modifie pas la définition des permissions associées aux rôles. Toute évolution de la matrice est contrôlée, relue et livrée séparément du formulaire d'affectation.	Cohérent avec la responsabilité de M1 et le seeder actuel.
R07	Un compte inactif ne peut plus ouvrir une nouvelle session ; une session déjà ouverte est invalidée à sa prochaine requête authentifiée.	Décision approuvée ; testée avec le middleware M1.
R08	La suppression d'un compte est logique et restaurable ; elle ne supprime ni ses rôles ni son historique d'audit.	Décision approuvée ; `SoftDeletes` est activé et aucune route de suppression définitive n'existe.
R09	Le dernier administrateur actif ne peut pas être rétrogradé, désactivé ou supprimé ; un administrateur ne peut pas désactiver ou supprimer son propre compte.	Décision approuvée ; les changements sensibles sont sérialisés par verrouillage du rôle administrateur.
R10	Les messages et journaux ne contiennent ni mot de passe, ni jeton de réinitialisation, ni cookie, ni identifiant de session. Les réponses ne révèlent pas de champs internes non nécessaires.	Les tests vérifient les accès, les réponses et l'absence de secrets dans les données exposées.
R11	Une inscription publique crée toujours un compte avec un seul rôle : `etudiant`. Le navigateur ne choisit jamais le rôle à la création.	Décision du demandeur, 24 octobre 2026 ; rôle étudiant déjà attribué par `RegisterStudent`.
R12	Le premier compte administrateur est créé ou promu via la commande `users:create-administrator`, qui demande les informations au moment de l'exécution et ne contient aucun mot de passe par défaut.	Décision du demandeur, 24 octobre 2026.

Matrice d'accès M2

La matrice ci-dessous applique la matrice M1 approuvée : seul le rôle `administrateur` dispose de `users.manage`.

Action M2	Étudiant	Responsable de classe	Administration	Administrateur
Consulter la liste administrative des comptes	Non	Non	Non	Oui
Consulter le détail administratif d'un compte	Non	Non	Non	Oui
Créer ou modifier un compte	Non	Non	Non	Oui
Affecter un rôle approuvé	Non	Non	Non	Oui
Activer, désactiver, supprimer logiquement ou restaurer un compte	Non	Non	Non	Oui, hors auto-désactivation/suppression et protection du dernier administrateur

Les utilisateurs restent autorisés à utiliser leurs propres parcours M1. La consultation ou modification du profil personnel n'est pas incluse par défaut dans M2.

6. Données, interfaces et états

Données

Donnée	Source / responsabilité	Contraintes à vérifier
Utilisateur (`users`)	Table et modèle existants ; cycle de vie administré par M2, authentification par M1	`name` et `email` existants ; e-mail unique ; mots de passe hachés ; `password` et `remember_token` masqués.
Rôle (`roles`)	Tables Spatie et `AccessControlSeeder`	Guard `web` ; rôle appartenant à la liste approuvée ; M2 ne crée ni ne supprime les rôles.
Affectation du rôle (`model_has_roles`)	Pivot Spatie existant	M2 impose un seul rôle par compte via `syncRoles` ; les rôles restent limités à la liste approuvée.
État du compte	Colonne `users.is_active`, valeur par défaut `true`.	La désactivation bloque la connexion et invalide les sessions à la requête authentifiée suivante.
Suppression logique (`users.deleted_at`)	Colonne ajoutée par la migration M2.	Les comptes supprimés peuvent être restaurés par un administrateur ; l'e-mail reste réservé au compte.
Audit (`user_audits`)	Table ajoutée par la migration M2.	Journal append-only des actions, acteur, compte ciblé et métadonnées minimales ; aucun mot de passe, jeton ou valeur personnelle avant/après n'est stocké. Rétention à définir.

Routes web implémentées

Les routes suivantes sont implémentées dans le groupe web/Inertia, derrière `auth` et `can:users.manage`.

Route	Accès	Résultat
`GET /admin/utilisateurs`	Authentifié et `users.manage`	Liste paginée des comptes, recherche et filtres validés.
`POST /admin/utilisateurs`	Authentifié et `users.manage`	Création avec un mot de passe aléatoire non communiqué, envoi du lien M1 de définition et affectation du rôle approuvé.
`PUT /admin/utilisateurs/{user}`	Authentifié et `users.manage`	Mise à jour validée, y compris l'affectation de rôle si autorisée.
`PATCH /admin/utilisateurs/{user}/statut`	Authentifié et `users.manage`	Activation ou désactivation du compte ; garde-fous contre l'auto-désactivation et la perte du dernier administrateur actif.
`DELETE /admin/utilisateurs/{user}`	Authentifié et `users.manage`	Suppression logique, avec conservation des pivots et de l'audit.
`PATCH /admin/utilisateurs/{user}/restaurer`	Authentifié et `users.manage`	Restauration d'un compte supprimé logiquement.

Ne pas introduire d'API JSON distincte tant que l'application utilise les conventions web/Inertia. Les formulaires doivent gérer les états chargement, erreur de validation, erreur réseau, succès et liste vide. En cas d'échec, conserver les champs non sensibles ; ne jamais repeupler un champ mot de passe.

7. Qualité, sécurité et risques

- Autoriser les opérations sur le serveur avec les permissions M1 ; tester les refus sur les requêtes directes.

- Valider et normaliser les données côté serveur ; s'appuyer sur la contrainte d'unicité en base comme protection finale contre les doublons concurrents.
- Utiliser une liste fermée de rôles récupérée côté serveur ; refuser un nom de rôle inconnu, un rôle hors guard ou une permission arbitraire.
- N'exposer dans les props Inertia que les attributs nécessaires ; vérifier que le modèle continue de masquer `password` et `remember_token`.
- Réutiliser la politique de mot de passe et les mécanismes de création/réinitialisation de M1 ; ne pas envoyer de mot de passe en clair par une voie non validée.
- Évaluer la pagination et l'indexation si le volume de comptes le nécessite ; aucun seuil de performance n'est actuellement défini.
- Ne pas journaliser les corps de requête sensibles ; l'audit M2 conserve seulement l'acteur, le compte ciblé, le type d'action et des métadonnées minimales sans valeurs personnelles avant/après.
- La création administrative dépend d'un transport de courriel fonctionnel pour transmettre le lien de définition du mot de passe. Le serveur SMTP a accepté un e-mail de test le 24 octobre 2026 ; la confirmation de sa réception en boîte reste à faire. L'opération échoue explicitement et annule la création si le lien ne peut pas être envoyé.
- La commande `user-audits:prune` supprime les entrées de plus de 12 mois et est planifiée chaque jour à 02:00 ; l'environnement déployé doit exécuter le planificateur Laravel (`php artisan schedule:run` chaque minute).

8. Questions ouvertes et décisions

ID	Question / décision attendue	Impact	Décideur attendu	État
Q01	Quels champs d'identité et de profil sont gérés ?	Formulaires, validation et données	Demandeur	Décidé : nom et e-mail uniquement, 24 octobre 2026
Q02	Un utilisateur possède-t-il exactement un rôle ou plusieurs rôles ?	Formulaire, stockage Spatie, tests et interprétation des permissions	Demandeur	Décidé : un rôle unique, remplaçable par l'administrateur, 24 octobre 2026
Q03	L'inscription publique est-elle maintenue ?	Route `/register`, navigation et attribution initiale	Demandeur	Décidé : oui ; tout nouvel inscrit devient `etudiant`, 24 octobre 2026. Remplace M1 D02
Q04	Comment l'utilisateur définit-il son premier mot de passe après création administrative ?	Parcours de création et dépendance e-mail	Demandeur / exploitation	Décidé : lien M1 de définition/réinitialisation ; nécessite un transport e-mail fonctionnel
Q05	Quel est l'effet d'une désactivation ?	Migration et intégration au mécanisme M1	Demandeur / sécurité	Décidé : bloquer les nouvelles connexions et invalider la session dès la prochaine requête authentifiée
Q06	Quelle politique de suppression est retenue ?	Historique, relations et conformité	Demandeur	Décidé : suppression logique et restauration ; pas de suppression définitive
Q07	Quelles protections s'appliquent à l'administrateur ?	Continuité d'administration et sécurité	Demandeur / sécurité	Décidé : protéger le dernier administrateur actif et interdire l'auto-désactivation
Q08	Faut-il conserver un historique des opérations sensibles ?	Journalisation, rétention et accès	Demandeur / exploitation	Décidé : audit des changements sensibles, conservés 12 mois puis purgés automatiquement
Q09	Les profils sont-ils rattachés à une classe, filière ou autre référentiel ?	Dépendance M3 et modèle utilisateur	Responsable métier / M3	À confirmer ; hors premier incrément

ID	Question / décision attendue	Impact	Décideur attendu	État
Q10	Quels volumes, filtres et objectifs de temps de réponse sont attendus pour la liste des comptes ?	Pagination, index et recette de performance	Responsable métier / exploitation	À confirmer
Q11	Quelle durée de conservation appliquer à l'audit ?	Politique de rétention et conformité	Demandeur	Décidé : 12 mois, puis purge automatique quotidienne, 24 octobre 2026

Les décisions principales de périmètre sont prises. Le rattachement pédagogique éventuel et les objectifs chiffrés de performance restent hors du premier incrément.

9. Étapes d'implémentation

Étape 0 — Vérifier et valider le cadrage

Statut : validée le 24 octobre 2026.

Objectif et prérequis

Confirmer que le périmètre de M2 est applicable au projet Academia et résoudre les décisions bloquantes. Prérequis : plan M1 disponible et accès au dépôt.

Travaux à réaliser et fichiers concernés

- Vérifier les migrations, le modèle `User`, le seeder, les routes d'authentification et les tests d'autorisation mentionnés dans la section 4.
- Consigner la décision de conserver l'inscription publique avec attribution automatique du rôle `etudiant` ; l'administrateur peut remplacer le rôle unique.
- Valider les champs nom/e-mail, le lien M1 de définition du mot de passe, la désactivation, la suppression logique, l'audit et la commande du premier administrateur.
- Réconcilier la règle d'inscription avec M1 par la décision D11.

Règles applicables et points de vigilance

R01 à R12. Les décisions du demandeur priment sur les propositions antérieures contradictoires.

Livrables

- Périmètre et matrice M2 approuvés.
- Questions ouvertes résolues ou explicitement reportées.
- Liste des écarts constatés entre le plan et le code.

Tests et cas limites

- Vérifier sur le papier un scénario de création autorisée, un refus sans `users.manage`, une adresse e-mail déjà utilisée et une affectation d'un rôle non autorisé.
- Vérifier le scénario attendu pour un compte désactivé et l'invalidation de sa session.

Critères de sortie

- [x] Les actions M2 et leurs acteurs sont validés.
- [x] La méthode de création du compte et de premier mot de passe est définie.

- [x] Le conflit sur l'inscription publique est traité dans M1 D11.
- [x] Les règles de rôles, suppression et état de compte nécessaires à l'étape suivante sont décidées.

Bilan d'exécution et validation

Bilan : le demandeur a confirmé l'inscription publique avec rôle étudiant par défaut, le rôle unique administrable, le profil nom/e-mail, le lien de définition de mot de passe, l'état actif, la suppression logique restaurable, l'audit, la protection du dernier administrateur et la commande de création du premier administrateur. D11 dans le plan M1 a été mise à jour pour refléter la décision. Base de test MySQL isolée choisie : `academia_test`.

Étape 1 — Préparer le modèle et les autorisations

Statut : validée techniquement le 24 octobre 2026.

Objectif et prérequis

Garantir un modèle de compte cohérent et des autorisations serveur avant d'exposer des écrans. Prérequis : étape 0 validée.

Travaux à réaliser et fichiers concernés

- Conserver les tables `users` et Spatie ; la migration M2 ajoute `is_active`, `deleted_at` et `user_audits`.
- Adapter `User` avec `SoftDeletes`, un état actif par défaut et les protections de sérialisation existantes.
- Protéger les routes par `users.manage`, limiter les rôles et imposer un rôle unique via `syncRoles`.
- Ajouter le middleware qui invalide l'accès d'un compte désactivé dès sa prochaine requête authentifiée.

Règles applicables et points de vigilance

R01 à R10 et R12. Ne pas créer de tables de rôles, permissions ou utilisateurs en doublon.

Livrables

- Modèle et migrations nécessaires, sans changement non approuvé.
- Contrôle serveur de la permission de gestion.
- Tests de règles d'accès et de validation.

Tests et cas limites

- L'administrateur avec `users.manage` est autorisé ; chaque rôle sans cette permission reçoit un refus.
- Une adresse e-mail dupliquée est refusée, y compris à la modification d'un autre compte.
- Les rôles inconnus, supprimés ou hors guard sont refusés.
- Les attributs sensibles ne sont pas exposés dans la sérialisation du compte.
- Vérifier la protection du dernier administrateur actif et le refus de l'auto-désactivation ou de l'auto-suppression.

Critères de sortie

- [x] Les migrations passent sur la base MySQL dédiée `academia_test`.
- [x] Les tests de permission, d'unicité et de validation réussissent.
- [x] Le modèle ne révèle aucun secret dans les réponses applicatives.

Bilan d'exécution et validation

Bilan : migrations MySQL M2 appliquées par les tests ; `User` supporte les comptes actifs et supprimés logiquement ; les tests Feature M1/M2 réussissent, dont refus d'accès, unicité, rôle unique, protection du dernier administrateur, création/restauration et purge des audits. Résultat consolidé : 37 tests, 413 assertions.

Étape 2 — Implémenter les opérations de gestion côté serveur

Statut : validée techniquement le 24 octobre 2026.

Objectif et prérequis

Fournir les opérations sécurisées de consultation, création et modification nécessaires à M2. Prérequis : étapes 0 et 1 validées.

Travaux à réaliser et fichiers concernés

- Ajouter les routes nommées, contrôleur et Form Requests Laravel/Inertia.
- Implémenter liste filtrable et paginée, création, modification du profil et remplacement du rôle.
- Ajouter activation/désactivation, suppression logique, restauration et journalisation.
- Envoyer le lien de définition du mot de passe après création administrative ; refuser la création si le service n'envoie pas de courriel.
- Ajouter `users:create-administrator`, qui crée ou promeut le compte initial sans mot de passe fixe.

Règles applicables et points de vigilance

R01 à R12. Vérifier l'autorisation sur toutes les routes et toutes les mutations. Ne pas faire confiance aux champs cachés ou à la sélection de rôle du navigateur.

Livrables

- Contrat web cohérent avec Inertia.
- Opérations de compte protégées et validations serveur.
- Tests Feature couvrant les cas nominaux, refus et conflits.

Tests et cas limites

- Création réussie avec rôle valide ; modification des données permises.
- Refus d'une demande par utilisateur non authentifié ou sans `users.manage`.
- Refus d'un rôle arbitraire et protection contre l'élévation de privilège par altération de la requête.
- E-mail en doublon, rôle inexistant, utilisateur inexistant et soumission invalide.
- Désactivation, réactivation, refus de connexion via M1 et invalidation d'une session déjà ouverte à sa prochaine requête authentifiée.
- Vérifier que les erreurs ne renvoient aucun mot de passe ni secret.

Critères de sortie

- [x] Chaque endpoint de gestion est autorisé côté serveur.
- [x] Les réponses ne contiennent que les attributs administratifs attendus.
- [x] Les tests Feature couvrent succès, validation, accès refusé et cas limites convenus.

Bilan d'exécution et validation

Bilan : six routes M2 enregistrées et protégées ; commandes, contrôleur et validations implémentés ; tests Feature M1/M2 passés sur MySQL (`37 tests`, `413 assertions`), dont la rétention automatique des audits.

Étape 3 — Réaliser les parcours de l'interface

Statut : validée dans le navigateur le 24 octobre 2026.

Objectif et prérequis

Permettre à l'administrateur de gérer les comptes sans contourner les règles serveur. Prérequis : contrat serveur de l'étape 2 validé.

Travaux réalisés et fichiers concernés

- Créer une page Inertia/Vue de gestion, avec liste, recherche, pagination, création, modification, rôles, états et audit récent.
- Afficher les erreurs serveur et confirmations sans données sensibles ; contrôler les rôles côté serveur.
- Ajouter les états de chargement, liste vide, erreurs de formulaire et confirmations.
- Relier le tableau de bord à la gestion de comptes pour les administrateurs.

Règles applicables et points de vigilance

R01 à R05 et R10 à R12. Le masquage des contrôles dans l'interface améliore l'expérience mais ne remplace jamais l'autorisation côté serveur.

Livrables

- Parcours de gestion des comptes intégrés à la navigation autorisée.
- Écrans cohérents avec les types et données retournés par le serveur.
- Tests d'interface pertinents selon l'outillage déjà configuré.

Tests et cas limites

- Parcours de création puis modification d'un compte.
- Erreurs de validation, doublon d'e-mail, rôle non valide, liste vide, perte réseau et nouvelle tentative.
- Vérifier qu'un utilisateur sans permission ne voit pas la gestion et reçoit toujours un refus s'il appelle directement les routes.
- Contrôler navigation clavier, focus et libellés des erreurs.

Critères de sortie

- [x] Les états de chargement, erreur, vide et succès sont traités.
- [x] Le typage, le lint, le formatage et le build de production passent.
- [x] Aucun secret ou attribut non autorisé n'est transmis à l'interface.
- [x] La recette navigateur confirme l'accès administrateur, la recherche, l'édition/annulation, le clavier et l'absence de débordement horizontal à 320 px et 1280 px.

Bilan d'exécution et validation

Bilan du 24 octobre 2026 : connexion puis accès administrateur vérifiés dans le navigateur sur MySQL isolé ; liste, recherche, formulaire d'édition et annulation contrôlés. À 1280 px et 320 px, aucun débordement horizontal du document ; le tableau reste défilable dans son conteneur sur mobile. Navigation clavier vérifiée entre le lien de retour et le premier champ ; le formulaire d'édition n'expose aucun champ mot de passe.

Étape 4 — Recetter et préparer la livraison

Statut : validée par le demandeur le 24 octobre 2026.

Objectif et prérequis

Prouver que le cycle de gestion respecte les règles métier et de sécurité convenues. Prérequis : étapes 1 à 3 terminées ; environnement de test isolé.

Travaux à réaliser et fichiers concernés

- Exécuter les tests ciblés M2 et les tests M1/autorisation touchés par les changements.
- Réaliser la recette de bout en bout avec des comptes et rôles de test.
- Vérifier migrations, seeders, données de démonstration et configuration avant déploiement.
- Mettre à jour ce plan avec les résultats réels, les écarts, les limitations et les décisions finales.

Règles applicables et points de vigilance

R01 à R12. Un build ou un test ciblé ne remplace pas la recette métier complète.

Livrables

- Résultats de recette documentés dans le bilan d'étape.
- Procédure de migration et limites connues, si des changements de schéma sont livrés.
- Validation métier explicite du périmètre implémenté.

Tests et cas limites

- Parcours complet : administrateur crée un compte, lui attribue un rôle approuvé, modifie ses données, puis un utilisateur non autorisé tente les mêmes opérations.
- Vérifier la connexion M1 avec le compte créé et l'application des permissions du rôle.
- Vérifier désactivation, invalidation de session à la requête suivante, suppression logique et restauration.
- Contrôler les journaux, props Inertia, réponses et formulaires pour détecter des secrets exposés.

Critères de sortie

- [x] Les tests ciblés M1/M2 et validations frontend ont été exécutés sur le code actuel.
- [x] Aucun défaut bloquant de contrôle d'accès ou de fuite de données n'a été relevé dans ces tests.
- [x] La recette navigateur de l'interface et de sa disposition mobile a été effectuée.
- [x] Le transport SMTP a accepté un e-mail de test envoyé à l'adresse configurée.
- [x] Le demandeur confirme la réception de l'e-mail d'essai.
- [x] Le demandeur valide le comportement livré.

Bilan d'exécution et validation

Bilan du 24 octobre 2026 : `php artisan test --compact tests/Feature` — 37 tests réussis, 413 assertions, sur la base MySQL dédiée `academia_test`. Le test de rétention couvre la suppression des entrées de plus de 12 mois et la conservation des entrées plus récentes ; `php artisan schedule:list` confirme la planification quotidienne à 02:00. `./vendor/bin/pint --test`, `npm run test:unit` (7 tests), `npm run lint`, `npm run typecheck`, `npm run format:check` et `npm run build` réussis. La recette navigateur a couvert l'accès autorisé, la recherche, l'édition/annulation, le clavier et le rendu à 1280 px et 320 px. Le serveur SMTP a accepté l'e-mail d'essai, et le demandeur a confirmé l'avoir reçu. Validation métier du M2 donnée par le demandeur le 24 octobre 2026.

10. Recette globale et critères d'acceptation

- **CA01 — Gestion autorisée :** un administrateur disposant de `users.manage` peut réaliser uniquement les opérations de compte approuvées.
- **CA02 — Refus serveur :** un utilisateur sans `users.manage` ne peut consulter ni modifier les comptes, même en appelant directement une URL ou en forgeant une requête.
- **CA03 — Rôle contrôlé :** l'affectation accepte uniquement les rôles de la matrice approuvée et n'accorde jamais de permissions arbitraires.

- **CA04 — Identité intègre :** une adresse e-mail déjà associée à un autre compte ne peut être utilisée à la création ou à la modification.
- **CA05 — Secrets protégés :** aucune réponse, page ou trace ne révèle le mot de passe, le jeton de réinitialisation, le cookie ou la session.
- **CA06 — Cohérence avec M1 :** une inscription publique reçoit le rôle étudiant ; le compte créé par un administrateur reçoit un lien de définition du mot de passe ; le remplacement de rôle prend effet à l'authentification suivante.
- **CA07 — État du compte :** une nouvelle connexion d'un compte désactivé est refusée et sa session active est invalidée à la requête authentifiée suivante.

11. Limites connues et consignes de livraison

- Le rattachement éventuel à M3 et les objectifs chiffrés de performance restent à confirmer.
- Le projet possède déjà l'authentification, la table `users`, les tables Spatie et un seeder des rôles. Ils doivent être réutilisés, pas recréés.
- La décision M1 D02 « pas d'inscription publique » a été remplacée par D11 : les inscriptions publiques sont étudiantes par défaut.
- L'envoi SMTP et la réception de l'e-mail d'essai ont été confirmés. La création administrative dépend du maintien d'un transport fonctionnel.
- L'audit ne stocke aucune valeur personnelle avant/après. Les entrées sont purgées après 12 mois par `user-audits:prune`, planifiée chaque jour à 02:00. L'environnement déployé doit exécuter le planificateur Laravel (`php artisan schedule:run` chaque minute).
- Aucune livraison ne doit être annoncée comme validée avant l'exécution réelle des tests et l'acceptation métier. Le déploiement reste une action distincte nécessitant une autorisation explicite.

12. Journal des décisions

Date	Décision	Responsable	Référence / justification
24 octobre 2026	Cadrage M2 approuvé : inscription publique avec rôle étudiant, rôle unique modifiable par l'administrateur, profil nom/e-mail, désactivation, suppression logique, audit et commande pour initialiser le premier administrateur.	Demandeur	Réponses au cadrage de l'étape 0.
24 octobre 2026	Tests configurés sur la base MySQL isolée `academia_test`, distincte de la base de développement.	Demandeur	Confirmation du moteur MySQL et choix du nom de base de test.
24 octobre 2026	Rétention d'audit fixée à 12 mois ; commande de purge et exécution quotidienne à 02:00 ajoutées et testées.	Demandeur	Q08 et Q11 ; test Feature et `php artisan schedule:list`.
24 octobre 2026	Recette navigateur terminée ; l'e-mail de test a été reçu et M2 a été validé.	Demandeur	Tests, recette navigateur, réception confirmée et acceptation métier.